

Problem Set 2, Part I

Problem 1: Variable scope

- 1) a, b, c
- 2) a, b, i, e
- 3) a, b, i, e, j, d
- 4) a, b
- 5) f, g
- 6) f, g

Problem 2: String objects and their methods

2-1

- a) `str2.substring(0, 11) + str1.substring(12)`
- b) `str2.substring(0, 3) + str1.substring(14,17)`
- c) `str1.substring(6, 12) + str2.toUpperCase().substring(11,17)`
- d) `str1.substring(str1.length()-3,str1.length()-2)`
- e) `str1.substring(str1.length()-7,str1.length()-8) + str2.substring(7,8)`
- f) `str1.length()`
- g)
`str1.substring(7,12)+str2.substring(0,11)+str2.substring(0,8).replace('i','o')`

Problem 3: Understanding code that uses an array

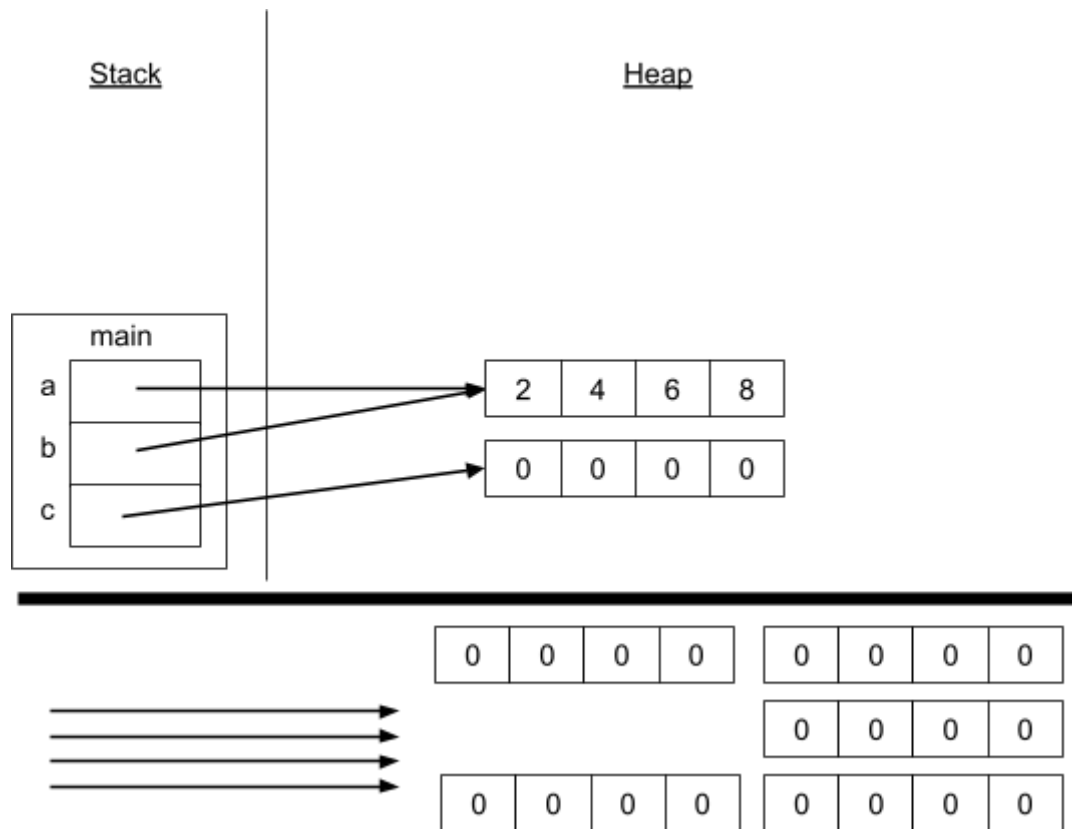
3-1)

i	val1	val2	arr
-	-	-	{1, 4, 9, 16, 25, 36, 49, 64}
0	4	9	{13, 4, 9, 16, 25, 36, 49, 64}
1	9	16	{13, 20, 9, 16, 25, 36, 49, 64}
2	16	25	{13, 20, 41, 16, 25, 36, 49, 64}
3	25	36	{13, 20, 41, 71, 25, 36, 49, 64}
4	36	49	{13, 20, 41, 71, 85, 36, 49, 64}
5	49	64	{13, 20, 41, 71, 85, 113, 49, 64}

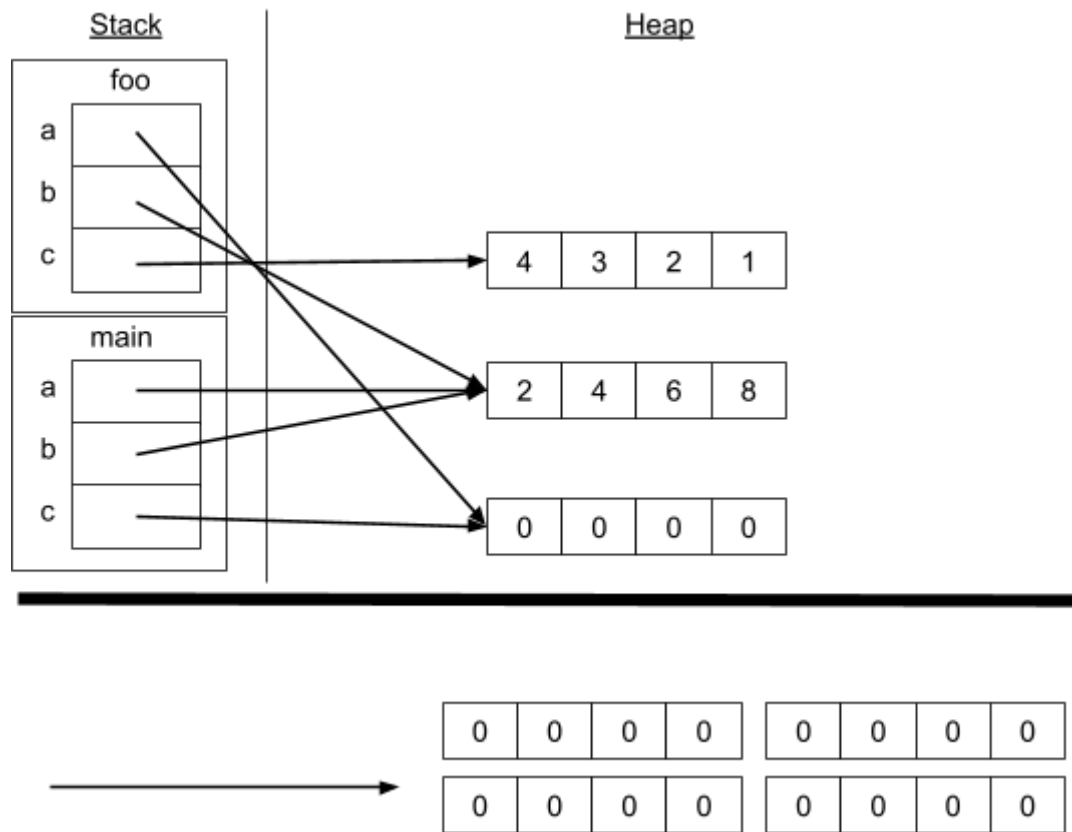
3-2)

No, we do not see the original array. As arrays are reference types in Java, meaning that the variable stores a pointer, address, or a reference, one might say, to its location within the memory. Whenever we pass those variables as parameters when calling a method, we are actually giving that address to the memory as a parameter, not a separate copy. So whenever the method changes the array, it changes it indefinitely as it changes it in the memory as well.

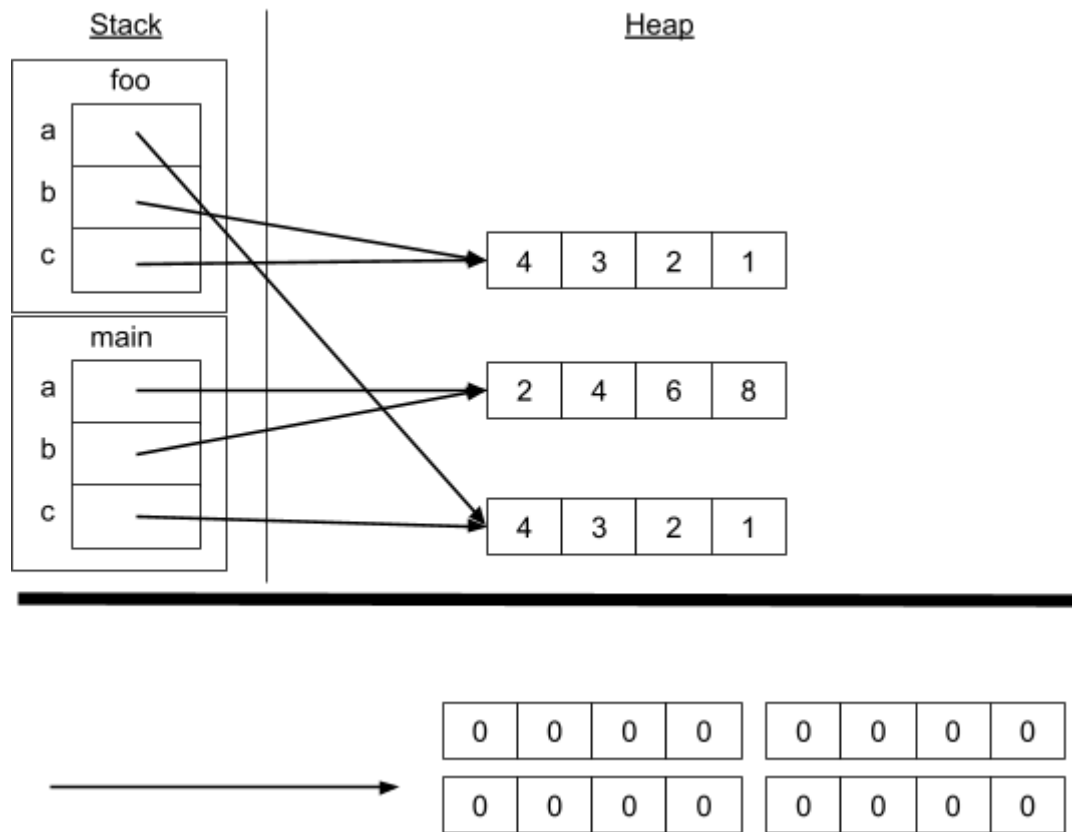
Problem 4: Memory management and arrays
4-1)



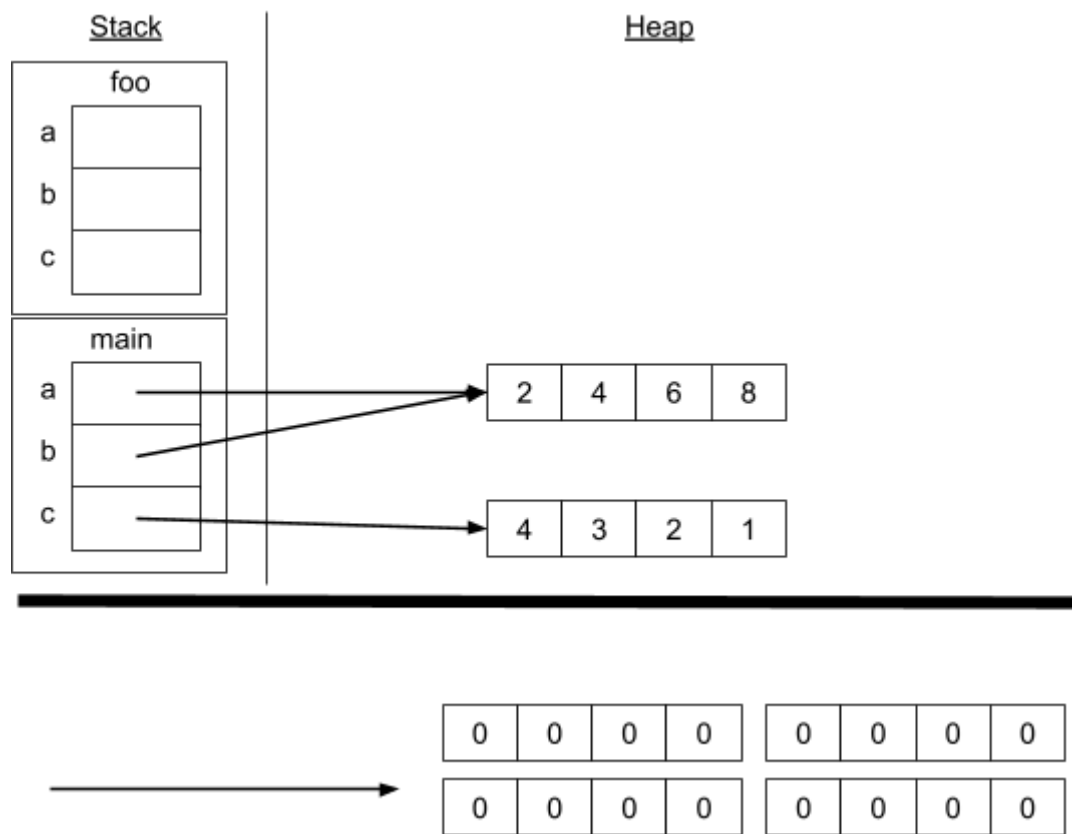
4-2)



4-3)



4-4)



Problem 5: Two-dimensional arrays

Java

```
int[][] twoD = {{16,12,8,4},
                {15,11,7,3},
                {14,10,6,2},
                {13,9,5,1}};
```

5-1)

Java

```
for(int i = 0; i<twoD.length;i++){
    for(int ii = 0; ii<twoD[i].length;ii++){
        if(twoD[i][ii] == 5)
            twoD[i][ii] = 20;
    }
}
```

5-2)

Java

```
for(int i = 0; i<twoD.length;i++){
    System.out.println(twoD[i][twoD[i].length]);
}
```

5-3)

Java

```
for(int i = twoD.length-1; i>=0;i--){
    for(int ii =0; ii<twoD[i].length; ii--){
        System.out.println(twoD[i][ii]);
    }
}
```